

Aggregate Functions

Summarizing Data with GROUP BY and HAVING

Lab Manual — Lab #4

Course	Database Systems
Topic	Aggregate functions (COUNT, SUM, AVG, MIN, MAX), GROUP BY, HAVING
Duration	One lab session (approx. 3 hours)
Tools	MySQL 8.x, MySQL Workbench / Command Line Client
Prerequisites	Lab #1 (Filters), Lab #2 (Joins), Lab #3 (Scalar Functions)
Builds On	WHERE filters, JOIN, scalar functions (used inside aggregates)

1. Learning Objectives

By the end of this lab, students will be able to:

- 1 Distinguish **scalar functions** (one value per row, covered last lab) from **aggregate functions** (one value per group).
- 2 Use the five core aggregate functions: **COUNT**, **SUM**, **AVG**, **MIN**, **MAX**.
- 3 Handle NULL values correctly when aggregating (COUNT(*) vs COUNT(col), SUM/AVG ignoring NULLs).
- 4 Use **COUNT(DISTINCT col)** to count distinct non-NULL values.
- 5 Use **GROUP BY** to compute aggregates per category.
- 6 Apply the **Golden Rule of GROUP BY**: every SELECT column must be in GROUP BY or inside an aggregate.
- 7 Use **HAVING** to filter groups, and explain why WHERE cannot do this.
- 8 Understand the logical execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.
- 9 Combine aggregates with JOINS to answer realistic business questions like 'total spend per customer' or 'best-selling category'.

Outcome

Each student will write SQL queries that summarize raw row-level data into meaningful per-category reports using aggregate functions, producing a single .sql file with all answers ready to be pushed to their GitHub repository.

2. Session Plan (3 Hours)

Time	Activity	Mode
0:00 – 0:10	Recap of Lab #3 + lab schema setup	Setup
0:10 – 0:30	What is aggregation? COUNT, SUM, AVG, MIN, MAX	Lecture + Demo
0:30 – 0:55	COUNT(*) vs COUNT(col) vs COUNT(DISTINCT); NULL handling	Lecture + Demo
0:55 – 1:25	Lab Tasks Part A — Whole-table aggregates	Hands-on
1:25 – 1:35	Short break	—
1:35 – 2:00	GROUP BY: per-category aggregation; the Golden Rule	Lecture + Demo
2:00 – 2:20	HAVING vs WHERE; query execution order	Lecture + Demo
2:20 – 2:35	Aggregates with joins — the real-world pattern	Lecture + Demo
2:35 – 2:55	Lab Tasks Part B — GROUP BY, HAVING, joins	Hands-on
2:55 – 3:00	Wrap-up, common mistakes, deliverables	Discussion

3. The Lab Schema — Retail Store Database

This lab uses a small **RetailStore** database with three related tables: *Customer*, *Product*, and *OrderItem* (which records each product sold to a customer). The data deliberately contains NULL values and a customer with no orders, so aggregates and LEFT JOIN + COUNT produce meaningful results.

3.1 Tables and Relationships

Table	Columns	Keys
Customer	CustID, CustName, City, JoinDate	PK: CustID
Product	ProdID, ProdName, Category, Price, StockQty	PK: ProdID
OrderItem	OrderID, CustID, ProdID, Quantity, OrderDate	PK: OrderID; FK: CustID; FK: ProdID

3.2 Setup Script — Run this First

```

CREATE DATABASE IF NOT EXISTS agg_lab;
USE agg_lab;

DROP TABLE IF EXISTS OrderItem, Product, Customer;

CREATE TABLE Customer (
    CustID    INT PRIMARY KEY,
    CustName  VARCHAR(60) NOT NULL,
    City      VARCHAR(30),
    JoinDate  DATE
);

CREATE TABLE Product (
    ProdID    INT PRIMARY KEY,
    ProdName  VARCHAR(60) NOT NULL,
    Category  VARCHAR(30),
    Price     DECIMAL(10,2),
    StockQty  INT
);

CREATE TABLE OrderItem (
    OrderID   INT PRIMARY KEY,
    CustID    INT,
    ProdID    INT,
    Quantity  INT,
    OrderDate DATE,
    FOREIGN KEY (CustID) REFERENCES Customer(CustID),
    FOREIGN KEY (ProdID) REFERENCES Product(ProdID)
);

-- Customers
INSERT INTO Customer VALUES
(1, 'Ali Khan',      'Lahore',      '2022-01-15'),
(2, 'Sara Iqbal',    'Karachi',    '2022-04-22'),
(3, 'Hamza Raza',    'Lahore',      '2023-02-10'),
(4, 'Ayesha Noor',   'Islamabad',   '2023-05-18'),
(5, 'Bilal Ahmed',   'Karachi',     '2023-09-01'),
(6, 'Fatima Sheikh', NULL,          '2024-01-12'),
(7, 'Usman Tariq',   'Lahore',      '2024-06-30'),
(8, 'Maira Javed',   'Islamabad',   '2024-08-25');
-- Note: customer 6 has NULL city; customer 8 will have no orders

```

```
-- Products
INSERT INTO Product VALUES
(101, 'Laptop Pro 15',      'Electronics', 185000.00, 12),
(102, 'Wireless Mouse',    'Electronics', 2500.00, 50),
(103, 'USB-C Cable',       'Electronics', 800.00, 100),
(104, 'Office Chair',      'Furniture', 18500.00, 8),
(105, 'Standing Desk',     'Furniture', 45000.50, 5),
(106, 'Notebook A4',       'Stationery', 350.00, 200),
(107, 'Ballpoint Pen 10pk', 'Stationery', 450.00, 150),
(108, 'Coffee Beans 1kg',  'Grocery', 1899.99, 30),
(109, 'Green Tea Box',     'Grocery', 650.00, 45),
(110, 'Bluetooth Speaker', 'Electronics', 7500.00, 18);

-- OrderItems (spread across 2023 and 2024)
INSERT INTO OrderItem VALUES
(1001, 1, 101, 1, '2023-03-10'),
(1002, 1, 102, 2, '2023-03-10'),
(1003, 2, 104, 1, '2023-05-22'),
(1004, 2, 106, 5, '2023-05-22'),
(1005, 3, 101, 1, '2023-08-15'),
(1006, 3, 110, 1, '2023-08-15'),
(1007, 4, 108, 3, '2023-11-02'),
(1008, 5, 103, 4, '2024-01-20'),
(1009, 5, 102, 1, '2024-01-20'),
(1010, 6, 105, 1, '2024-02-14'),
(1011, 7, 107, 2, '2024-04-08'),
(1012, 7, 106, 10, '2024-04-08'),
(1013, 7, 109, 3, '2024-07-19'),
(1014, 2, 110, 1, '2024-09-05'),
(1015, 3, 108, 2, '2024-10-11');

-- Customer 8 (Maira) has no orders yet
```

Deliberate gaps in the data

Customer 6 (Fatima) has a NULL city — useful for showing how COUNT(City) excludes NULLs.

Customer 8 (Maira) has no orders — useful for LEFT JOIN + COUNT to find customers with zero activity. Stationery products (107) sell heavily in small quantities; Electronics (101) sell in low quantities at high prices — different categories tell different stories when aggregated.

4. What Is Aggregation?

A **scalar function** (from Lab #3) operates on one row and returns one value per row. An **aggregate function** operates on many rows and **collapses them into a single value**.

Function type	Input	Output	Example
Scalar (Lab #3)	One row at a time	One value per row	UPPER(CustName)
Aggregate (this lab)	A set of rows	One value for the whole set	AVG(Price)

Used alone, an aggregate produces a **single summary row** for the entire result set. Combined with **GROUP BY**, it produces one row per category.

Class Example 1 — Whole-table aggregate (one row result):

```
SELECT COUNT(*) AS Total, AVG(Price) AS AvgPrice
FROM Product;
-- Result: ONE row regardless of how many products exist
```

5. The Five Core Aggregate Functions

Function	What it does	NULL handling
COUNT(*)	Number of rows	Counts every row, including those with NULLs
COUNT(col)	Number of non-NULL values in col	Skips NULLs
COUNT(DISTINCT col)	Number of distinct non-NULL values	Skips NULLs and duplicates
SUM(col)	Total of numeric column	Skips NULLs
AVG(col)	Arithmetic mean of numeric column	Skips NULLs (denominator may shrink)
MIN(col) / MAX(col)	Smallest / largest value	Skips NULLs; works on numbers, dates, strings

Class Example 2 — Whole-table customer summary:

```
SELECT COUNT(*)           AS NumCustomers,    -- 8
       COUNT(City)        AS WithCity,        -- 7 (Fatima has NULL)
       COUNT(DISTINCT City) AS DistinctCities  -- 3 (Lahore, Karachi, Islamabad)
FROM Customer;
```

Class Example 3 — Product price statistics:

```
SELECT COUNT(*)           AS Items,
       MIN(Price)         AS Cheapest,
       MAX(Price)         AS Priciest,
       ROUND(AVG(Price),2) AS AvgPrice,
       SUM(StockQty)      AS TotalStock
FROM Product;
```

COUNT(*) vs COUNT(col)

COUNT(*) counts *rows*. COUNT(col) counts rows where *col* is not NULL. For our Customer table, COUNT(*) returns 8 but COUNT(City) returns 7 (Fatima has NULL city). Picking the wrong form is one of the most common SQL bugs.

Class Example 4 — Using MIN/MAX on dates and strings:

```
SELECT MIN(JoinDate) AS EarliestJoin,  
       MAX(JoinDate) AS LatestJoin,  
       MIN(CustName) AS AlphabeticallyFirst, -- 'Ali Khan'  
       MAX(CustName) AS AlphabeticallyLast   -- 'Usman Tariq'  
FROM Customer;
```

6. GROUP BY — Aggregates Per Category

Without GROUP BY, an aggregate collapses the whole table into one row. With GROUP BY, the rows are first split into groups by the GROUP BY columns, and the aggregate is computed for each group separately. The result has one row per group.

Class Example 5 — Number of customers per city:

```
SELECT City, COUNT(*) AS NumCustomers
FROM Customer
GROUP BY City;
-- Result:
-- Lahore      -> 3
-- Karachi     -> 2
-- Islamabad   -> 2
-- NULL        -> 1 (Fatima with NULL city forms its own group)
```

Class Example 6 — Average price per product category:

```
SELECT Category,
       COUNT(*)           AS NumProducts,
       ROUND(AVG(Price), 2) AS AvgPrice,
       MIN(Price)         AS Cheapest,
       MAX(Price)         AS Priciest
FROM Product
GROUP BY Category
ORDER BY AvgPrice DESC;
```

Class Example 7 — Grouping by multiple columns:

```
-- One row per (city, year-of-join) combination
SELECT City,
       YEAR(JoinDate) AS JoinYear,
       COUNT(*)       AS NumCustomers
FROM Customer
GROUP BY City, YEAR(JoinDate)
ORDER BY City, JoinYear;
```

Golden Rule of GROUP BY

Every column in the SELECT list must either appear in the GROUP BY clause or be inside an aggregate function. Writing `SELECT CustName, COUNT(*) FROM Customer GROUP BY City` is logically broken — `CustName` is not in the GROUP BY and is not aggregated. MySQL may return an arbitrary `CustName` from each group; other databases would reject the query outright.

7. HAVING — Filtering Groups

WHERE filters rows *before* aggregation. **HAVING** filters groups *after* aggregation. You cannot put an aggregate function inside a WHERE clause because the aggregates have not been computed yet at that stage of query execution.

Class Example 8 — Wrong vs right way:

```
-- WRONG: aggregates are not allowed in WHERE
SELECT Category, AVG(Price) FROM Product
WHERE AVG(Price) > 5000          -- ERROR
GROUP BY Category;
```

```
-- CORRECT: use HAVING for conditions on aggregates
SELECT Category, AVG(Price) AS Avg
FROM Product
GROUP BY Category
HAVING AVG(Price) > 5000;
```

Class Example 9 — Cities with more than 1 customer:

```
SELECT City, COUNT(*) AS NumCustomers
FROM Customer
WHERE City IS NOT NULL           -- row-level filter BEFORE grouping
GROUP BY City
HAVING COUNT(*) > 1             -- group-level filter AFTER aggregation
ORDER BY NumCustomers DESC;
```

Class Example 10 — WHERE and HAVING together:

```
-- Categories whose products in stock (qty > 0) have avg price > 1000
SELECT Category,
       COUNT(*)           AS NumInStock,
       ROUND(AVG(Price), 2) AS AvgPrice
FROM Product
WHERE StockQty > 0         -- step 1: filter individual rows
GROUP BY Category         -- step 2: form groups
HAVING AVG(Price) > 1000   -- step 3: filter groups
ORDER BY AvgPrice DESC;
```

When to use WHERE vs HAVING

If the condition can be checked on a single row (e.g. *StockQty > 0*, *City = 'Lahore'*), put it in **WHERE** — it runs before grouping and is faster. If the condition involves an aggregate (e.g. *COUNT(*) > 1*, *AVG(...)* > 500), put it in **HAVING** — that's the only place aggregates can appear in a filter.

8. The Logical Execution Order of a Query

You write a query in this order: **SELECT** → **FROM** → **WHERE** → **GROUP BY** → **HAVING** → **ORDER BY**. MySQL **executes** it in a different order. Understanding this prevents many beginner mistakes.

Step	Clause	What happens
1	FROM / JOIN	Tables are loaded and joined.
2	WHERE	Rows are filtered (row-level filter). Aliases not yet defined.
3	GROUP BY	Remaining rows are split into groups by the GROUP BY columns.
4	HAVING	Groups are filtered (group-level filter). Aggregates available here.
5	SELECT	Columns and aggregates are computed; aliases assigned.
6	ORDER BY	Result is sorted (aliases CAN be used here).
7	LIMIT	Top N rows are returned.

Why SELECT aliases don't work in WHERE

SELECT runs at step 5, but WHERE runs at step 2. By the time WHERE is evaluated, the alias does not exist yet. The same logic applies to GROUP BY and HAVING (in standard SQL — MySQL is sometimes lenient). The safe rule: aliases work in **ORDER BY**, repeat the full expression elsewhere.

9. Aggregates With JOINS — The Most Common Real-World Pattern

The combination **JOIN + GROUP BY + Aggregate** answers questions like 'total spend per customer' or 'best-selling category'. This is where you'll spend most of your time in real reporting work.

Class Example 11 — Total spend per customer:

```
SELECT c.CustID,
       c.CustName,
       COUNT(o.OrderID)           AS NumOrders,
       SUM(o.Quantity * p.Price) AS TotalSpend
FROM   Customer c
LEFT JOIN OrderItem o ON c.CustID = o.CustID
LEFT JOIN Product p ON o.ProdID = p.ProdID
GROUP BY c.CustID, c.CustName
ORDER BY TotalSpend DESC;
-- LEFT JOIN ensures customer 8 (Maira, no orders) still appears
-- For Maira: NumOrders = 0, TotalSpend = NULL
```

Class Example 12 — Top categories by total revenue:

```
SELECT p.Category,
       COUNT(o.OrderID)           AS NumOrders,
       SUM(o.Quantity * p.Price) AS Revenue
FROM   OrderItem o
JOIN    Product p ON o.ProdID = p.ProdID
GROUP BY p.Category
ORDER BY Revenue DESC;
```

Class Example 13 — Best customer in each city (with HAVING):

```
-- Cities whose customers collectively spent more than 50,000
SELECT c.City,
       COUNT(DISTINCT c.CustID) AS NumCustomers,
       SUM(o.Quantity * p.Price) AS CityRevenue
FROM   Customer c
JOIN   OrderItem o ON c.CustID = o.CustID
JOIN   Product p ON o.ProdID = p.ProdID
GROUP BY c.City
HAVING SUM(o.Quantity * p.Price) > 50000
ORDER BY CityRevenue DESC;
```

LEFT JOIN before aggregation is the safe default

If you want every customer to appear (including those with zero orders), use LEFT JOIN. With INNER JOIN, customers with no matching OrderItem rows disappear entirely from the result — their COUNT will not even show as 0. Be explicit about whether you want to see 'zero activity' groups.

10. Lab Tasks

Solve each task in MySQL using the RetailStore schema from section 3. Save all queries in a single file named `RollNo_Aggregates.sql`, with a comment line before each answer (e.g. `-- Task A1`).

Part A — Whole-Table Aggregates & COUNT Variations

#	Task
A1	Count the total number of customers, products, and orders in the database (three separate queries or one with three columns).
A2	Find the cheapest and most expensive products. Show MinPrice and MaxPrice.
A3	What is the average price of all products? Round to 2 decimal places.
A4	What is the total stock quantity across all products?
A5	How many distinct cities do customers live in (ignoring NULL)?
A6	How many distinct categories of products are there?
A7	How many customers have a recorded city? How many do not? (Two queries.)
A8	Find the earliest and latest order date in the system.
A9	Compute the total revenue from all orders. Revenue per row = Quantity * Price; total = SUM of all rows. (Hint: JOIN OrderItem with Product.)
A10	Compute the average quantity per order (average of Quantity column in OrderItem). Round to 2 decimals.

Part B — GROUP BY, HAVING, and Aggregates with JOINS

#	Task
B1	Number of customers in each city. Sort by count descending. (GROUP BY.)
B2	Number of products in each category. Sort by count descending.
B3	For each product category, find the average, minimum, and maximum price. Sort by average price descending.
B4	Total stock quantity per category. Sort by total descending.
B5	Number of orders placed in each year. Show Year and NumOrders, sorted by year. (Hint: use YEAR() from Lab #3.)
B6	Number of orders placed each month of 2024. Show Month (1–12) and NumOrders, sorted by month.
B7	Find product categories where the average price is greater than 5,000. (GROUP BY + HAVING.)
B8	Find cities with more than 1 customer (exclude NULL city). (WHERE + GROUP BY + HAVING.)
B9	For each customer, count how many orders they have placed. Include customers with zero orders. (LEFT JOIN + COUNT.)
B10	Total quantity sold for each product. Show ProdName and TotalQty, sorted by TotalQty descending. Include products that have never been sold (TotalQty should be 0 or NULL). (LEFT JOIN.)
B11	Total revenue (Quantity * Price) per product category. Sort by revenue descending.
B12	For each customer, compute their total spend. Show CustName and TotalSpend, sorted descending. Include customers with no orders. (LEFT JOIN.)
B13	List customers whose total spend exceeds 50,000 . Show CustName and TotalSpend. (HAVING.)
B14	For each city, count the number of customers and total revenue generated by them. Include only cities with more than 1 customer. (GROUP BY + HAVING.)
B15	Find the top 3 best-selling products by total quantity sold. (GROUP BY + ORDER BY + LIMIT.)
B16	For each year, compute the total revenue. Show Year and Revenue, sorted by year.
B17	Find the average order value (revenue per order). Group by OrderID first to get each order's value, then average those values. (Two-step thinking — you may use the AVG of SUM trick or a subquery.)

Debugging tip

If a GROUP BY query gives the wrong total: comment out the aggregate first and run the raw join with all rows. Verify the row count matches your expectation. Wrong totals usually mean a join is multiplying rows (e.g. INNER vs LEFT, or a missing join condition).

11. Assessment Problem (Solve Individually)

Scenario — University Database. A university stores its data in three tables: Student, Course, and Enrollment. Build the schema in MySQL, load the provided sample data, and answer the twelve questions that follow. *This problem is graded.*

11.1 Schema

Table	Columns	Keys
Student	StudentID, FullName, City, EnrollDate	PK: StudentID
Course	CourseID, CourseName, Department, Credits, Fee	PK: CourseID
Enrollment	EnrollID, StudentID, CourseID, Marks, EnrollmentDate	PK: EnrollID; FK: StudentID; FK: CourseID

11.2 Setup Script

```
CREATE DATABASE IF NOT EXISTS uni_lab;
USE uni_lab;

DROP TABLE IF EXISTS Enrollment, Course, Student;

CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    FullName   VARCHAR(60) NOT NULL,
    City       VARCHAR(30),
    EnrollDate DATE
);

CREATE TABLE Course (
    CourseID   VARCHAR(10) PRIMARY KEY,
    CourseName VARCHAR(60) NOT NULL,
    Department VARCHAR(30),
    Credits    INT,
    Fee        DECIMAL(10,2)
);

CREATE TABLE Enrollment (
    EnrollID      INT PRIMARY KEY,
    StudentID     INT,
    CourseID      VARCHAR(10),
    Marks         INT,
    EnrollmentDate DATE,
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);

INSERT INTO Student VALUES
(1001, 'Ahmad Raza', 'Lahore', '2022-09-01'),
(1002, 'Sara Imran', 'Karachi', '2022-09-01'),
(1003, 'Bilal Khan', 'Lahore', '2023-09-01'),
(1004, 'Fatima Ali', 'Islamabad', '2022-09-01'),
(1005, 'Hira Yousaf', NULL, '2024-09-01'),
(1006, 'Zain Abbas', 'Karachi', '2023-09-01'),
(1007, 'Mehwish Anwar', 'Lahore', '2022-09-01'),
(1008, 'Talha Hussain', 'Islamabad', '2024-09-01'),
(1009, 'Areeba Yasin', 'Lahore', '2023-09-01');
-- Student 1009 has not enrolled in any course yet
```

```
INSERT INTO Course VALUES
('CS101','Intro to Programming', 'Computer Science', 3, 25000),
('CS201','Database Systems',      'Computer Science', 3, 28000),
('CS301','Operating Systems',     'Computer Science', 4, 30000),
('MT101','Calculus I',            'Mathematics',      3, 22000),
('EE201','Digital Logic',         'Electrical Engg',  3, 26000),
('BB301','Marketing Basics',     'Business',         3, 24000);
-- Course BB301 has no enrollments
```

```
INSERT INTO Enrollment VALUES
(1, 1001, 'CS101', 78, '2022-09-15'),
(2, 1001, 'CS201', 85, '2023-09-15'),
(3, 1001, 'MT101', 90, '2022-09-15'),
(4, 1002, 'CS101', 65, '2022-09-15'),
(5, 1002, 'CS201', 72, '2023-09-15'),
(6, 1003, 'CS101', 88, '2023-09-15'),
(7, 1003, 'EE201', 80, '2023-09-15'),
(8, 1004, 'MT101', 95, '2022-09-15'),
(9, 1004, 'CS201', 70, '2023-09-15'),
(10, 1005, 'CS101', 55, '2024-09-15'),
(11, 1006, 'CS101', 82, '2023-09-15'),
(12, 1006, 'CS301', 76, '2024-09-15'),
(13, 1007, 'CS201', 91, '2023-09-15'),
(14, 1007, 'CS301', 86, '2024-09-15'),
(15, 1008, 'CS101', 60, '2024-09-15'),
(16, 1008, 'MT101', 68, '2024-09-15');
```

11.3 Questions

#	Question	Marks
Q1	How many students and how many courses are there? (Two values in one query.)	5
Q2	How many distinct cities do students come from (ignore NULL)?	5
Q3	Compute the average, minimum, and maximum Marks across all enrollments.	10
Q4	Number of students in each city. Sort by count descending. Include the NULL-city group at the end.	8
Q5	Number of courses offered by each department, sorted by count descending.	8
Q6	For each course, show CourseName, the number of students enrolled, and the average marks. Sort by average marks descending. (JOIN + GROUP BY.)	12
Q7	List courses with an average marks above 80 . Show CourseName and AvgMarks. (HAVING.)	10
Q8	Total fee revenue per department, assuming each enrolled student paid the course fee. Show Department and TotalRevenue, sorted descending. (3-table join + GROUP BY.)	12
Q9	For each student, show how many courses they are enrolled in and their average marks. Include students with no enrollments (count = 0, avg = NULL). (LEFT JOIN + GROUP BY.)	10
Q10	Find students who scored above 85 in at least one course. Show their FullName and the highest mark they have achieved. (GROUP BY + HAVING + MAX.)	10
Q11	List departments where the average marks across all their courses is below 75. Show Department and OverallAvgMarks. (Multi-step JOIN + GROUP BY + HAVING.)	5
Q12	Find the top 3 students by total fee they have paid (sum of fees of their enrolled courses). Show FullName and TotalFee. (JOIN + GROUP BY + ORDER BY + LIMIT.)	5

11.4 Submission

- Push your RollNo_Aggregates.sql file to your existing GitHub repository in a folder named Lab4-Aggregates/.
- The file must contain: the setup script (or a comment confirming you ran it), all Lab Task answers (Parts A and B), and all 12 Assessment answers, each labelled with a comment (e.g. -- Q5).
- Share the GitHub link to that folder before the deadline announced in class.

12. Common Mistakes to Avoid

- Putting an aggregate function inside a WHERE clause. Aggregates only work in HAVING (or in the SELECT list after grouping).
- Forgetting the Golden Rule of GROUP BY: every SELECT column must be in GROUP BY or inside an aggregate. MySQL may silently return arbitrary values; other databases will error.
- Confusing COUNT(*) with COUNT(col) — they differ when the column has NULLs.
- Forgetting DISTINCT inside COUNT when you want unique values: COUNT(City) counts rows, COUNT(DISTINCT City) counts unique cities.
- Using INNER JOIN with aggregation when you wanted to include 'zero-activity' groups. Use LEFT JOIN, and remember the count for those groups will be 0 (because COUNT(col) skips NULL).
- Counting NULL rows by accident: COUNT(*) counts every row including those that became NULL through a LEFT JOIN. Use COUNT(o.OrderID) instead.
- Trying to reference a SELECT alias in WHERE or HAVING. WHERE runs before SELECT; HAVING runs before SELECT in standard SQL. Repeat the expression instead.
- Using GROUP BY without an aggregate, expecting it to deduplicate — that's what **DISTINCT** is for.
- Mistaking row count for distinct count: SELECT COUNT(City) FROM Customer may return more than the number of cities because duplicates are counted.
- Forgetting that AVG skips NULLs in the denominator — the average of (10, NULL, 20) is 15, not 10.

13. Appendix — Quick Reference

```
-- THE FIVE CORE AGGREGATES
COUNT(*)           -- number of rows (including NULLs)
COUNT(col)         -- number of non-NULL values in col
COUNT(DISTINCT col) -- number of distinct non-NULL values
SUM(col)            -- total of numeric column (NULLs skipped)
AVG(col)            -- mean of numeric column (NULLs skipped)
MIN(col) / MAX(col) -- smallest/largest (works on number, date, string)

-- WHOLE-TABLE AGGREGATE (one row result)
SELECT COUNT(*), AVG(Price), MAX(Price)
FROM Product;

-- GROUP BY (one row per group)
SELECT category, COUNT(*) AS n, ROUND(AVG(price), 2) AS avg_p
FROM product
GROUP BY category
ORDER BY avg_p DESC;

-- GROUP BY multiple columns
SELECT city, YEAR(join_date) AS yr, COUNT(*) AS n
FROM customer
GROUP BY city, YEAR(join_date);

-- WHERE + GROUP BY + HAVING + ORDER BY + LIMIT
SELECT category, COUNT(*) AS n, AVG(price) AS avg_p
FROM product
WHERE stock_qty > 0           -- row-level filter (BEFORE grouping)
GROUP BY category
```

```
HAVING COUNT(*) >= 3          -- group-level filter (AFTER aggregation)
ORDER BY avg_p DESC
LIMIT 10;

-- AGGREGATE + LEFT JOIN (keep groups with zero matching rows)
SELECT c.CustName,
       COUNT(o.OrderID)      AS num_orders,
       SUM(o.Quantity * p.Price) AS total_spend
FROM Customer c
LEFT JOIN OrderItem o ON c.CustID = o.CustID
LEFT JOIN Product p ON o.ProdID = p.ProdID
GROUP BY c.CustID, c.CustName
ORDER BY total_spend DESC;

-- LOGICAL ORDER: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT
```

End of Lab Manual #4 — Aggregate Functions, GROUP BY, HAVING.